

Table of Contents

Introduction	3
Task 1 – Environment Setup & Data Modeling	3
Task 2 – Data Import & Document Design	5
Task 3 – CRUD Operations	7
1. Create Queries	7
2. Read Queries	10
3. Update Queries	13
4. Delete Queries	15
Query D1:	15
Conclusion	16

Introduction

Today's ecommerce applications require databases to process a huge volume of searches and dynamic product data. These requirements may cause issues in traditional relational databases (RDBMS) as they are built with static table schemas and can involve multiple JOINS. A popular NoSQL database, MongoDB addresses these issues by storing data as flexible BSON documents. The full implementation of an e-commerce Database system with flexible data modeling, data set validation, schema validation and CRUD operations on the system using Mongo Shell.

Task 1 – Environment Setup & Data Modeling

The database was created using ecommerce_portfolio as its identifier.

```
> use ecommerce_portfolio
< switched to db ecommerce_portfolio
> show collections
<
> db.createCollection("customers")
  db.createCollection("products")
  db.createCollection("orders")
< { ok: 1 }
ecommerce_portfolio>
```

Data Modeling Strategy: Embed vs Referencing:

The hybrid approach to data modelling improved the performance as well as the accuracy of the data. The method depends on the type of data, and embeds and references at the same time.

Embedded Data ('orders.items'):

Data about items in an order are embedded in the order document. This includes the item's price, quantity and other information at the time of purchase. Embedding allows for quicker retrieval of a full order, without having to join on multiple collections and does not modify order information if the product price changes after order placed.

Referenced Data ('customers, products'):

Customers and products are kept in separate collections. Information about customers can change over time. For instance, the mail address as well as home delivery address might be altered. By using the reference of customer_id, the same customer data is not stored more than once. The 'product_id' references are used to update product information and stock, without changing the older order records.

Schema Validation

For customers collection:

```
{
  "name": { "$type": "string" }
}
```

For products collection:

```
{
  "name": { "$type": "string" }
}
```

For orders collection:

```
{
  "status": { "$type": "string" }
}
```

A schema validation was set up for all three collections in MongoDB Compass. The Validation tab was opened for the Customers collection and a Rule was created to ensure that the name field includes a string. For the products collection, the name field was also obligatory and had to be a string, as above. A validation rule was added to the orders collection to make sure that the status field is a string. Each rule was added to the changes and, after adding each rule, the changes were saved so that the validation requirements would be applied.

Task 2 – Data Import & Document Design

The provided datasets, customers.json, products.json, and orders.json, were imported into the mongodb-ecommerce-project database using the mongoimport command.

```
> db.customers.countDocuments()
db.customers.find().limit(3)

db.products.countDocuments()
db.products.find().limit(3)

db.orders.countDocuments()
db.orders.find().limit(3)
< {
  _id: 'ORD00001',
  customer_id: 'CUST0013',
  order_date: '2024-02-11',
  status: 'Returned',
  payment_method: 'Credit Card',
  shipping_city: 'Sylhet',
  items: [
    {
      product_id: 'PROD0061',
      product_name: 'Action Figure Set',
      category: 'Toys & Games',
      unit_price: 30.78,
      quantity: 2,
      line_total: 61.56
    }
  ],
  total_amount: 61.56
}
```

```

}
{
  _id: 'ORD00002',
  customer_id: 'CUST0014',
  order_date: '2024-09-01',
  status: 'Shipped',
  payment_method: 'bKash',
  shipping_city: 'Houston',
  items: [
    {
      product_id: 'PROD0031',
      product_name: 'Ceramic Dinner Set',
      category: 'Home & Kitchen',
      unit_price: 425.62,
      quantity: 1,
      line_total: 425.62
    },
    {
      product_id: 'PROD0043',
      product_name: 'Yoga Mat',
      category: 'Sports & Outdoors',
      unit_price: 358.2,
      quantity: 5,
      line_total: 1791
    }
  ],
  total_amount: 2216.62
}

```

```

}
{
  _id: 'ORD00003',
  customer_id: 'CUST0020',
  order_date: '2024-06-20',
  status: 'Delivered',
  payment_method: 'PayPal',
  shipping_city: 'Dhaka',
  items: [
    {
      product_id: 'PROD0050',
      product_name: 'Herbal Shampoo 400ml',
      category: 'Beauty & Personal Care',
      unit_price: 377.97,
      quantity: 2,
      line_total: 755.94
    }
  ],
  total_amount: 755.94
}

```

Enterprise 22204158> |

Inter-Collection Relationships & Reference Architecture:

Data model is relational referencing model, it is used to ensure consistency of data in e-commerce system; to avoid duplication of data. The three collections all share the three explicitly named identifiers below:

Customer-to-Order Relationship ('customer_id'):

The primary collection "orders" has a normalized one-to-many (1:N) relationship with the collection "customers" by the relationship `order.customer_id → customers._id`. The customer information (such as names, e-mail addresses or membership levels) doesn't exist hundreds of times in the order documents; rather, each order reference points to an ID of the customer who has the information. This design guarantees that any change made to a customer's personal information will be seen in the central collection of orders, instead of having to update existing order documents.

Product-to-Order Relationship ('product_id'):

The documents in the orders array have an embedded array called 'items[]' containing the items that were purchased. Each sub document will have a 'product_id' to create a many-to-many (N:M) relationship between 'orders.items.product_id' → 'products._id'. Most of the core product catalog information remains in the collection 'products', such as inventory (global) stock_quantity, catalog (category) and supplier. Meanwhile, the number of items purchased and the price per item are recorded in the order in the embedded array of items, and a historical snapshot of the transaction is maintained.

Task 3 – CRUD Operations

1. Create Queries

Query C1:

```
db.customers.insertOne({
  _id: "CUST333",
  name: "Lima Tawsar",
  email: "ltiw@example.com",
  city: "Dhaka",
  membership_tier: "Gold",
  join_date: "2026-06-01",
  is_active: true
});
```

Output:

```
{ acknowledged: true, insertedId: 'CUST333' }
```

```
> db.customers.insertOne({
  _id: "CUST333",
  name: "Lima Tawsar",
  email: "ltiw@example.com",
  city: "Dhaka",
  membership_tier: "Gold",
  join_date: "2026-06-01",
  is_active: true
});
< {
  acknowledged: true,
  insertedId: 'CUST333'
}
Enterprise 22204158>
```

Interpretation: It adds a new active customer to the database.

Query C2:

```
db.orders.insertOne({  
  
  id: "ORD333",  
  
  customer_id: "CUST333",  
  
  order_date: "2026-06-05",  
  
  status: "Pending",  
  
  total_amount: 1500,  
  
  items: [{ product_id: "PROD010", quantity: 2, price_per_unit: 750 }]  
});
```

Output:

```
{  
  
  acknowledged: true, insertedId: ObjectId('6aa19f99ad408fddf754f750')  
  
}
```

```

> db.orders.insertOne({
  id: "ORD333",
  customer_id: "CUST333",
  order_date: "2026-06-05",
  status: "Pending",
  total_amount: 1500,
  items: [{ product_id: "PROD010", quantity: 2, price_per_unit: 750 }]
});
< {
  acknowledged: true,
  insertedId: ObjectId('6aa19f99ad408fddf754f750')
}

```

Interpretation: It adds a new order for CUST333 with PROD010 which is included in the order.

2. Read Queries

Query R1

```

db.products.find(
  { price: { $gt: 100 }, category: { $in: ["Electronics", "Stationery"] } },
  { name: 1, price: 1, category: 1, _id: 0 });

```

Output: This query shows products, such as wireless mouse, bluetooth speaker and USB-C charger, which are above \$100. It shows product name, price and category.

```

> db.products.find(
  { price: { $gt: 100 }, category: { $in: ["Electronics", "Stationery"] } },
  { name: 1, price: 1, category: 1, _id: 0 }
);
< {
  name: 'Wireless Mouse',
  category: 'Electronics',
  price: 128.52
}
{
  name: 'Bluetooth Speaker',
  category: 'Electronics',
  price: 442.65
}
{
  name: 'USB-C Charger',
  category: 'Electronics',
  price: 463.25
}
{
  name: 'Smartwatch',
  category: 'Electronics',
  price: 441.3
}
{
  name: 'Noise Cancelling Headphones',
  category: 'Electronics',
  price: 392.28
}

```

Interpretation: It creates an inventory report by separating products according to price and it shows only necessary data in order to keep the data output smaller.

Query R2

```

db.orders.find(
  { "items.product_id": "PROD010" },
  { customer_id: 1, items: 1, total_amount: 1 }
);

```

Output: It returns the order details of product PROD010. For example, the order ORD333 is made by CUST333.

```

> db.orders.find({ "items.product_id": "PROD010" },
  { customer_id: 1, items: 1, total_amount: 1 }
);
< {
  _id: ObjectId('6aa19f99ad408fddf754f750'),
  customer_id: 'CUST333',
  total_amount: 1500,
  items: [
    {
      product_id: 'PROD010',
      quantity: 2,
      price_per_unit: 750
    }
  ]
}

```

Interpretation: It searches the order items in order to find orders where PRD010 was bought.

Query R3

```

db.customers.find(
  { email: { $regex: "^I", $options: "i" } },
  { name: 1, email: 1, _id: 0 }
);

```

Output: It matches customer accounts' emails starting with “I”.

```

> db.customers.find(
  { email: { $regex: "^l", $options: "i" } },
  { name: 1, email: 1, _id: 0 }
);
< {
  name: 'Linda Smith',
  email: 'linda.smith1@example.com'
}
{
  name: 'Linda Alam',
  email: 'linda.alam21@example.com'
}
{
  name: 'Linda Rahman',
  email: 'linda.rahman25@example.com'
}

```

Interpretation: It helps to search customers by using different text patterns and it makes smoother to find particular customer accounts.

3. Update Queries

Query U1:

```

db.orders.updateOne(
  { id: "ORD333" },
  { $set: { status: "Delivered" } }
);

```

Output:

```

{
  acknowledged: true, matchedCount: 1, modifiedCount: 1
}

```

```

> db.orders.updateOne(
  { id: "ORD333" },
  { $set: { status: "Delivered" } }
);
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

Interpretation: It changes the order status from pending to delivered.

Query U2:

```

db.customers.updateOne(
  { _id: "CUST333" },
  { $set: { city: "Chittagong" } }
);

```

Output:

```

{
  acknowledged: true, matchedCount: 1, modifiedCount: 1
}

```

```

> db.customers.updateOne(
  { _id: "CUST333" },
  { $set: { city: "Chittagong" } }
);
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

Interpretation: It updates the address of CUST333 when the customer changes his/her location.

4. Delete Queries

Query D1:

```
db.orders.deleteOne({ id: "ORD333" });
```

Output:

```
{
  acknowledged: true,
  deletedCount: 1
}
```

```
> db.orders.deleteOne({ id: "ORD333" });
< {
  acknowledged: true,
  deletedCount: 1
}
```

Interpretation: It deletes order ORD333 from the orders collection.

Query D2:

```
db.customers.deleteOne({ _id: "CUST333" });
```

Output:

```
{
  acknowledged: true, deletedCount: 1
}
```

```
> db.customers.deleteOne({ _id: "CUST333" });  
< {  
  acknowledged: true,  
  deletedCount: 1  
}
```

Interpretation: It deletes the temporary record of CUSR333 in order to keep the collection clean and organized.

Conclusion

This assignment defines the schema design of a NoSQL document oriented application for e-commerce websites using MongoDB. It uses a hybrid modelling approach (embedded array of order line items, referenced collection of customers and products) to improve read performance on the data while maintaining transactional integrity and no data duplication. By being multi-collection, it ensures that historical transactions won't be affected by the changes of the dynamic profile. Also, the ability to efficiently project on fields, with \$gt and \$in as examples of filters that can be applied to the array elements, as well as the ability to match with a regular expression, were demonstrated through various CRUD operations, showing the analytical power of MongoDB. Overall, document databases are fast and effective for e-commerce workloads that are semi-structured and have high velocity. And aids in conquering with a good overview.